

Assignment Report

Name: Asser Mohammed Khairallah

Group Code: GIZ5_SWD5_S2

Assignment: Day 13 - Entity Framework Core, Code-First, and Mapping Strategies

Part 01: Theoretical Questions

1. Why do we use Code First rather than DB First approach in Entity Framework Core?

- Code First treats the C# domain classes (POCOs) as the single source of truth. EF Core's runtime model-building engine reads these classes via reflection at application startup, builds an internal object model (`IModel`), and generates the database schema from it through Migrations. This keeps the schema under source control alongside the code, allows the entire data layer to evolve through versioned, repeatable Migration files, and removes the need to manually keep a separate `.edmx` or database diagram synchronized with the classes. DB First reverses this flow by scaffolding classes from an existing database, which is useful for legacy databases but couples the domain model tightly to whatever schema already exists, offering less control over naming, relationships, and architectural boundaries.

2. Why do we prefer using Fluent API over Data Annotations for entity configurations?

- Data Annotations are attributes applied directly on entity properties (e.g., `[Required]` , `[MaxLength(50)]`), which means configuration logic is hard-coded into the domain model itself, violating separation of concerns and limiting the entity class to only what attributes expose. Fluent API, configured inside `OnModelCreating` (typically through `IEntityTypeConfiguration<T>` classes), operates directly against EF Core's model-building API, giving access to configuration options that have no attribute equivalent at all, such as composite keys, shadow properties, query filters, table splitting, and complex relationship fine-tuning (cascade behavior, multiple foreign keys to the same table). Fluent API also keeps persistence concerns out of the POCO classes entirely, preserving a cleaner separation between the domain model and the EF Core mapping layer.
-

Part 02: Additional Concepts

1. Explain the role of EF Core Migrations and how Convention, Data Annotations, and Fluent API mapping strategies interact during schema generation.

- EF Core Migrations are the mechanism that translates changes in the model into incremental, versioned SQL scripts. Each migration captures a snapshot diff of the model (stored in a generated `Designer.cs` snapshot file) and produces `Up()` / `Down()` methods that apply or revert the corresponding DDL statements (`CREATE TABLE` , `ALTER TABLE` , etc.) against the database via the configured provider.
- Before migrations run, EF Core must first build the final in-memory model, which is resolved through three layers evaluated in a strict precedence order:
 - **Convention-based configuration:** The baseline. EF Core infers mapping automatically from naming and type patterns (e.g., a property named `Id` becomes the primary key; a `string` becomes `nvarchar(max)` by default).
 - **Data Annotations:** Override conventions at the property/class level, applied next in the model-building pipeline.
 - **Fluent API:** Has the highest precedence and always overrides both Conventions and Data

Annotations when conflicts occur, since it is applied explicitly and last inside `OnModelCreating`.

- This layered resolution allows developers to rely on conventions for simple, default-shaped entities, use Data Annotations for quick validation-style constraints, and fall back to Fluent API only where precise, low-level control over the generated schema is required.